

Time Complexity Analysis Practice Sheet

Part 1: Recurrence Relations

1. Recurrence Relation 1

$$T(n) = 2T(n/2) + n$$

Identify the time complexity of the recurrence relation.

2. Recurrence Relation 2

$$T(n) = T(n/3) + 1 + \log n + n^{1/2}$$

Determine the time complexity.

3. Recurrence Relation 3

$$T(n) = 4T(n/2) + n^2 + 8n + 3n + 100$$

- Find the time complexity of this recurrence.

4. Recurrence Relation 4

$$T(n) = 3T(n/4) + n \log n$$

Calculate the time complexity.

5. Recurrence Relation 5

$$T(n) = 2T(n/2) + n \log n + n^{1/2}$$

What is the time complexity of the recurrence?

6. Derive the recurrence relation for the code samples below. Solve the recurrence relation to find the time complexity.

```
a. int func1(int n) {  
    if (n <= 1) {  
        return 1;  
    }  
    return func1(n - 1) + func1(n - 1);  
}
```

```
b. int func2(int n) {  
    if (n <= 1) {  
        return 1;  
    }  
    return func2(n / 2) + func2(n / 2) + n;  
}
```

```
c. int func3(int n) {  
    if (n <= 1) {  
        return 1;  
    }  
    int sum = 0;  
    for (int i = 0; i < n; i++) {  
        sum += func3(i);  
    }  
    return sum;  
}
```

```
d. int func4(int n) {  
    if (n <= 1) {  
        return 1;  
    }  
    return 2 * func4(n / 2) + n;  
}
```

```
e. int func5(int n) {  
    if (n <= 1) {  
        return 1;  
    }  
    int sum = 0;  
    for (int i = 0; i < n; i++) {  
        sum += func5(n - 1);  
    }  
    return sum;  
}
```

f. function func6(n):

```
    if n <= 1:
        return 1
    a = func6(n - 1)
    b = func6(n - 1)
    return a + b
```

g. function func7(n):

```
    if n <= 1:
        return 1
    a = func7(n / 3)
    b = func7(n / 3)
    c = func7(n / 3)
    return a + b + c + n
```

h. function func8(n):

```
    if n <= 1:
        return 1
    sum = 0
    for i = 1 to n:
        sum = sum + func8(i)
    return sum
```

i. function func9(n):

```
    if n <= 1:
        return 1
    return 3 * func9(n / 2) + n^2
```

j. function func10(n):

```
    if n <= 1:
        return 1
    return func10(n - 1) + func10(n - 2) + func10(n - 3)
```

Part 2: Loop Problems

Determine the time complexity.

```
1. for (int i = 0; i < sqrt(n); i++) {  
    // Constant time operation  
}
```

```
2. z=0  
for (int i = 0; z < n; i++) {  
    z=z+i;  
    // Constant time operation  
}
```

```
3. 1. for (int i = 0; i*i < n; i++) {  
    // Constant time operation  
}
```

```
4. for (int i = 0; i < n; i++)  
{ // Constant time operation }
```

```
5. n=8  
for (int i = 0; i < n; i=i+8)  
{ // Constant time operation }
```

```
6. for (int i = 1; i < n; i *= 2)  
{ // Constant time operation }
```

```
7. for (int i = 0; i < n; i++)  
    { for (int j = 1; j < n; j *= 2)  
      { // Constant time operation } }
```

```
8. for (int i = 0; i < n; i++)
```

```
{ for (int j = 0; j < n; j++)  
{ // Constant time operation } }
```

```
9. for (int i = 0; i < n; i++)  
  for (int j = 0; j < n; j++)  
  { for (int k = 0; k < n; k++)  
    { // Constant time operation } } }
```

```
10. for (int i = 0; i < sqrt(n); i++)  
  { for (int j = 0; j < sqrt(n); j++)  
    { // Constant time operation } }
```

```
11. for (int i = 1; i < n; i *= 2)  
  { for (int j = 1; j < n; j *= 2)  
    { // Constant time operation } }
```

```
12. for (int i = 0; i < n; i++)  
  { for (int j = 0; j < sqrt(n); j++)  
    { for (int i = 0; i < n; i=i+n)  
      { // Constant time operation } } }
```

```
13. for (int i = 0; i < n; i++)  
  { // Constant time operation  
    for (int j = 1; j < n; j *= 2)  
      { // Constant time operation }  
    // Linear operation  
    for (int k = 0; k < n; k++)  
      { // Constant time operation } }
```